

# MORE PROGRAMS OR MORE ROLLS?

## SEPARATING COVERAGE FROM SPECIALIZATION IN LLM HARNESSES

**Anonymous authors**

Paper under double-blind review

### ABSTRACT

Do generated LLM harnesses add task-specific abilities, or merely more opportunities to produce correct answers? We separate coverage, repeatability, and usable selection using same-code controls on 386 fully observed MATH-500 tasks. Eight generated programs plus a baseline are compared with nine byte-identical baseline slots, each executed three times. The clones produce 2.16 percentage points of repeat-averaged oracle headroom despite zero program specialization. Generated programs show much stronger residual score-pattern correlation (0.789 versus 0.006), but persistent losses to the baseline span 100 tasks, whereas persistent wins occur on only one. The frozen feature selector gains zero; at 27 executions, both arms reach 98.70% oracle coverage. Held-out-repeat evidence leaves stable complementarity unresolved. Supporting BIRD traces distinguish mechanisms that are absent, untriggered, or executed but broken. Thus, in the tested panel, strong repeatability yields neither a gain for the frozen selector nor greater full-budget oracle coverage than additional executions of the same program.

## 1 INTRODUCTION

Should a fixed inference budget buy more programs or more executions of one program? Automated agent design makes the first option increasingly accessible: a builder writes a population of LLM harnesses, each specifying prompts, tools, and control flow around a frozen solver (Hu et al., 2024; Nie et al., 2026; Zhang et al., 2026b). The appeal is that different programs might handle different tasks better. To justify that interpretation, however, a population’s gains must be separated from the extra attempts used to evaluate it.

The difficulty is that different observed successes do not require different abilities. Two independent executions of a program with success probability 0.5 cover a task with probability 0.75. No program specialization has been introduced. Pass@ $k$  measures coverage over multiple samples; self-consistency and sampling-and-voting ensembles aggregate multiple attempts into one answer (Chen et al., 2021; Wang et al., 2022; Li et al., 2024). An oracle that selects a correct answer after seeing all outcomes can therefore show a population gain even when every member has the same code. Averaging a few repeats does not automatically remove this ambiguity: choosing the largest estimated success rate also favors sampling noise.

We ask what evidence distinguishes this coverage opportunity from task specialization. Three questions organize the study. Did the recorded executions contain additional correct answers? Does a member’s advantage on a task persist in a fresh execution? Can a rule choose usefully before seeing the answer, and improve on spending the same resources on a fixed program? These questions concern different quantities. A positive answer to the first need not answer the other two.

We first report single-run coverage in the full population of 35 generated MATH-500 programs, then test the specialization interpretation on a selected subset. The central control holds population size fixed and changes whether the programs differ: eight generated harnesses plus single-call are compared with nine byte-identical baseline slots. Three fresh repeats let us discover task-wise choices on two executions and score them on the third. The same-code arm measures what this procedure reports without program specialization. On the same panel, a selector frozen on development tasks then tests selection from question features, and an execution-matched oracle replay compares avail-

|                                                                                                                                                            |                                                                                                                                                          |                                                                                                                                               |
|------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1. Coverage</b><br>Repeat-mean oracle headroom<br>Generated: 2.33 pp<br>Same code: 2.16 pp<br><b>Identical programs also produce positive headroom.</b> | <b>2. Repeatability</b><br>Residual score correlation<br>Generated: 0.789<br>Same code: 0.006<br>Persistent losses: 100 tasks<br>Persistent wins: 1 task | <b>3. Selection and replay</b><br>Frozen selector gain<br>0.00 pp<br>Oracle coverage, 27 executions<br>Generated: 98.70%<br>Same code: 98.70% |
|------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|

Figure 1: **Positive headroom and strong repeatability coexist with zero selector gain and equal full-budget oracle coverage.** All panels use the same 386 MATH tasks, eight generated programs plus single-call, and nine identical single-call slots, with three fresh executions per member. Persistent losses and wins are relative to single-call in all three repeats. Selector gain is relative to the development-selected fixed baseline. Replay compares the generated panel with same-code executions using post-execution correctness at matched harness execution counts.

able coverage. Finally, BIRD traces examine whether code differences actually reach execution, providing a complementary diagnosis within the SQL setting.

The evidence yields three contributions. **First**, identical programs produce positive repeat-averaged oracle headroom, so that signal alone cannot identify program specialization. **Second**, generated programs have substantially more repeatable scored-outcome patterns than clones, dominated by persistent losses: losses span 100 tasks (570 member–task pairs), whereas persistent wins occur on one task. Stable differences therefore need not provide useful task advantages; the clone-adjusted held-out gain is  $-0.26$  pp. **Third**, the frozen selector gains 0.00 pp, and the 27-execution oracle replay ties at 98.70%. On this panel, more programs produce neither a gain for the tested selector nor greater full-budget oracle coverage than more executions of the baseline.

## 2 RELATED WORK

**Automated agent and harness design.** ADAS treats agent programs as a search space (Hu et al., 2024). Self-Harness mines failures from traces, proposes targeted edits, and retains edits after regression testing (Zhang et al., 2026b); TTHE adapts the executable harness during evaluation using unlabeled traces (Nie et al., 2026). Other systems improve task adaptation and validation allocation (Zhang et al., 2026a; Miyai et al., 2026), or make harness behavior observable and editable through verification, diagnosis, localization, and policy evolution (Xu et al., 2026; Chen et al., 2026a; Lin et al., 2026; Luo et al., 2026; Wang et al., 2026; Chen et al., 2026b). El et al. (2025) find low diversity in task-level score patterns among the automatically designed agents they study. We separate two explanations for population-level results: distinct programs can execute the same behavior, and identical programs can produce different outcomes. Trace inspection tests whether proposed mechanisms execute; repeated same-code controls isolate the coverage available without changing the program. This connects program equivalence and mutation testing (Budd & Angluin, 1982; Papadakis et al., 2019) to performance attribution in generated harness populations.

**Repeated sampling and ensembles.** Pass@ $k$  measures success within  $k$  samples; self-consistency and identical-agent ensembles aggregate multiple attempts (Chen et al., 2021; Wang et al., 2022; Li et al., 2024); even nominally temperature-0 calls can vary (Ouyang et al., 2023). We use these sampling gains as an explicit reference for generated-program complementarity at matched execution budgets. Post-execution verifiers rank candidate solutions (Kwok et al., 2026), while ensemble selection chooses a model combination on validation data and averages its predictions at inference (Caruana et al., 2004). Both assess or combine generated outputs, whereas our selector chooses a program before executing it. Our oracle replay and frozen selector retain this distinction.

**Selection and resource allocation.** HELIX uses complementary sibling trajectories both to expand verified coverage and to supply data for subsequent model improvement (Fan & Huang, 2026).

STS compares six reasoning paradigms and trains an embedding-based router to recover part of their oracle gap (Zhou et al., 2026). Our study evaluates generated programs with the target model fixed, separating oracle coverage, repeat-stable task advantages, and realized selector gain. The same-code population supplies the sampling reference for that separation. Harness sensitivity across model–problem pairs (Gupta et al., 2026) and across evaluated models (Cho, 2026), search routing (Kim, 2026), and preference-trained model routing (Ong et al., 2024) connect this question to algorithm selection and portfolios (Rice, 1976; Xu et al., 2008; Kotthoff, 2014). GRASP instead selects edits to a bounded skill library under regression constraints (Moll et al., 2026). The distinction between quantitative and qualitative interaction (Gail & Simon, 1985) explains why varying effect sizes need not change the best action. Our contribution is an evaluation that measures all three quantities in the same population and shows where they diverge.

### 3 PROBLEM FORMULATION

A population can supply more correct answers without containing task specialists, and specialists can exist without an effective selector. We formalize these distinctions before describing the controls. Let  $P$  contain baseline  $H_0$ , and let  $Y(h, x, r) \in \{0, 1\}$  denote member  $h$ ’s correctness on task  $x$  in repeat  $r$ . Accuracies and frequencies use %; gains and headroom use percentage points (pp). For example, a probability difference of 0.04145 is reported as 4.15 pp after rounding.

**Coverage: were additional correct answers available?** A one-run oracle sees correctness before choosing a member. Its headroom over the best fixed member in the recorded matrix is

$$H(P) = \frac{1}{n} \sum_{i=1}^n \max_{h \in P} Y(h, x_i, r) - \max_{h \in P} \frac{1}{n} \sum_{i=1}^n Y(h, x_i, r). \quad (1)$$

This measures available answers, including successes from repeated sampling. The repeat-averaged plugin  $\hat{H}$  replaces  $Y$  by each member–task mean. With few repeats, maximizing those means can still select favorable estimation errors;  $\hat{H} > 0$  is not by itself evidence of specialization.

**Stable complementarity: does the best member change with the task?** To remove execution noise conceptually, write  $p_h(x) = \Pr\{Y(h, x, r) = 1\}$  under a fixed execution condition and define  $H_{\text{stable}} = \mathbb{E}_x \max_h p_h(x) - \max_h \mathbb{E}_x p_h(x)$ . For two members with  $\Delta(x) = p_1(x) - p_0(x)$ ,

$$H_{\text{stable}} = \frac{1}{2} (\mathbb{E}|\Delta(x)| - |\mathbb{E}\Delta(x)|). \quad (2)$$

If one member is better everywhere, the expression is zero, even when the size of its advantage varies. Positive complementarity requires task-wise crossover. Identical programs under the same condition have equal  $p_h(x)$  and hence zero stable complementarity. This population identity clarifies the target; it does not estimate unknown probabilities from a small number of executions.

**Usable selection: can the advantage inform a choice?** A selector  $\pi_Z$  must choose from task features  $Z(x)$  before seeing the result. Even stable crossovers may not be predictable from those features. Learning a choice from earlier outcomes on the *same* task instead tests repeat generalization, a different information setting. We evaluate both separately. Practical resource allocation adds one more comparison: a selection gain over one fixed execution must be assessed against additional fixed-program executions under a common budget and answer-selection rule. Equal numbers of harness executions need not entail equal model calls or tokens.

## 4 EXPERIMENTAL DESIGN

### 4.1 HARNESS POPULATIONS AND EVALUATION SETUP

A *builder* writes a Python harness; a *target solver* answers its model calls; an external *judge* scores its final output. The reported comparisons use GLM-5.3-Flash (Appendix A.3). Each harness implements `solve(question)` and calls `self.llm`; BIRD also exposes schema and SQL feedback through `self.execute`. Gold answers are available only to the judge. The single-call baseline makes one temperature-0 call. Generated programs can change prompts, decoding, sample count, and control flow. These choices determine each member’s computation and sampling behavior. Table 1 identifies the role of each study.

| Study               | Programs                                        | Evaluation                                   | Question answered                   |
|---------------------|-------------------------------------------------|----------------------------------------------|-------------------------------------|
| MATH exploration    | 35 generated + single-call                      | 386 tasks; one run                           | Is there error coverage?            |
| MATH repeat control | 8 generated + single-call, versus 9 clones      | Same 386 tasks; 3 fresh repeats              | Do choices transfer across repeats? |
| BIRD discovery      | 12 generated + single-call + <code>react</code> | 60 tasks; one run                            | What mechanisms execute?            |
| BIRD admission      | 6 builders $\times$ 3 seeds $\times$ 4 arms     | 1,169 for primary contrast; 400 for all arms | Does screening change coverage?     |

Table 1: **Each study addresses a different inference.** One run means one execution per member-task pair. The repeated MATH panel is selected from the exploratory population. BIRD supplies separate execution and admission evidence, with effects reported by study.

**MATH populations and panel selection.** Six builders—GLM-5.3, Qwen3.8-Max, DeepSeek-V4-Pro, Kimi-K3, MiniMax-M3, and ERNIE-5.0-Thinking-Preview—receive eight slots each under one seed. Each slot is an independent generation opportunity, not an assigned strategy. Prompts request an improvement over one greedy call and leave program structure to the builder, without a cross-slot diversity constraint. Retries reuse the same prompt without feeding back earlier code or evaluation outcomes. Each slot allows three attempts at most (temperature 0.7; 32,768 output tokens per attempt), stopping at the first program that passes parsing, import, interface, and nonempty-answer checks on two development questions. This yields 35 frozen programs from 48 slots for subsequent evaluation. Appendix A.1 summarizes the builder instruction and its interface constraints.

For the repeat panel, a historical mean-call ceiling of 3.05 leaves 21 eligible members; a fixed salted hash selects one per builder and two at-large seats. This favors low-call programs. Selected sources include draft-check-revise, agreement checking, and sample voting; Appendix A.2 lists them. Adding single-call gives nine members. Both this panel and nine identical baseline slots receive three fresh, cache-disabled repeats, with decoding fixed by each member’s source.

From MATH-500 (Lightman et al., 2023), we use 100 development problems and a common analysis set of 386 evaluation problems. We determine this set after collection by requiring complete observations in both repeat arms and the single-run population; 14 problems with incomplete repeat observations are excluded. All reported MATH primary comparisons use these same task identities, with 100% observed scoring cells within the analysis set. The repeat study uses fresh executions of these tasks. Appendix B reports the exclusions. The judge extracts the final answer, preferring the requested `####` line, then compares numeric values when both parse and normalized LaTeX otherwise. Appendix A.1 details the comparator and its audit.

**BIRD discovery and admission.** BIRD (Li et al., 2023) requires SQL for a question and database. TTHE’s proposer with a GLM-5.3-Flash builder produces the 12 discovery candidates, admitted by import and smoke-execution checks. They and two supplied baselines are evaluated on 60 questions. The separate admission study crosses free versus assigned strategies with ungated versus conformance-gated admission, using six builders, three seeds, and eight slots per arm. At most three attempts per slot are allowed (temperature 0.7; 16,384 output tokens). Assigned strategies include execution repair, three-sample voting, and schema linking. All arms require a valid interface and nonempty SQL; gated arms also check mechanism conformance and exclude some structures.

The admission split reserves 365 questions in two databases for development and 1,169 in nine databases for evaluation; all four arms use a pre-drawn 400-question core. An 18-item smoke test is a disclosed development-exposure exception. Predicted and gold query results are compared using the official set-of-rows scorer with a 30-second timeout and failures scored zero. Discovery instead uses an earlier stringified-row proxy with a row cap. Admission also uses shared response caching; the MATH repeats use cache-disabled execution. These conditions and the bundled nature of the gating intervention are retained in Appendices E.1 and F.

## 4.2 SAME-CODE CONTROLS AND REPEAT VALIDATION

**Same-code slots isolate the sampling explanation.** Running more members introduces more attempts as well as more code. We therefore compare nine distinct members with nine byte-identical single-call slots, each evaluated three times per task. The latter preserve the opportunity for stochastic outcomes while removing program specialization, providing an equal-ability reference.

**Held-out repeats separate discovery from validation.** Choosing a task’s best program and validating it on the same outcomes would reuse lucky successes. Instead, each of three folds uses two discovery repeats to freeze both a per-task member mapping and a single best fixed member. The third repeat measures their accuracy difference. Averaging folds gives  $G_{\text{real}}$  and  $G_{\text{clone}}$ ; the fixed comparator is selected by discovery accuracy. We then compare the arms on their common complete tasks:

$$D = G_{\text{real}} - G_{\text{clone}}. \quad (3)$$

Here the subtraction is of paired task-level gains. It asks whether past outcomes identify choices that transfer to another execution better for distinct programs than for identical ones. The diagnostic measures cross-repeat transfer on the same evaluation tasks.

**Complete observations and sensitivity govern interpretation.** Every primary quantity, including both descriptive plugins and the two cross-fitted gains, uses the same 386 complete tasks. Each arm contains 10,422/10,422 scored member–task–repeat cells (100% completeness); all estimates use recorded binary scores. The original acquisition-coverage gate is assessed before subsetting and is passed: 3.50% of tasks were incomplete, below its 10% ceiling. Positive-effect checks, including  $D > 1$  pp, and the minimum of 30 paired tasks remain unchanged. Known-truth simulations assess repeat sensitivity. Appendix B documents selection into the analysis set; Appendix C gives the rule, tie handling, uncertainty, and sensitivity analyses. The acquisition protocol preceded collection; the v3 procedure and common-complete-set analysis were determined afterward.

**Recovery snapshot and exploratory repeatability.** We analyze the timestamped recovery snapshot in Appendix B, with programs, solver, tasks, and repeat count held fixed. We additionally examine cross-repeat correlation after removing additive member and task effects. This exploratory correlation measures residual score-pattern repeatability, while  $G$  measures held-out selection gain. Construction and scoring details appear in Appendix B.1.

## 4.3 FROZEN SELECTION AND EXECUTION-MATCHED REPLAY

**Feature-based selection before execution.** The repeat diagnostic uses past outcomes on the same task; a practical selector must instead choose before seeing an answer. We train  $\pi_Z$  on the 100 development tasks using TF-IDF question features and member accuracies computed within training folds. Five-fold task-level cross-validation keeps all repeats of a task together. The policy is frozen before evaluation and falls back to single-call when its top-two probability gap is below 0.15; ties use fixed member-ID order. The five fold-model probability vectors are averaged before selection. Its inputs consist of question features and development-derived member statistics. Comparators include the development-selected fixed member and one randomly drawn fixed member.

**Oracle replay after execution.** A separate offline comparison allocates  $b \in \{3, 9, 15, 27\}$  harness executions to a development-ranked portfolio and to the clone arm. Each included member contributes three repeats; the clone reference uses subsets of its 27 recorded executions. Both arms select with post-execution correctness, measuring available oracle coverage. One harness execution can contain several model calls; model-call counts and token costs vary by program. Appendix D specifies the frozen policy, replay, and accounting.

For recovery analysis, the previously recorded selector choices and portfolio order stay fixed. Replay uses only tasks complete in both arms and computes the exact uniform average over clone subsets. Portfolio members follow the recorded development ranking.

## 5 RESULTS

### 5.1 SINGLE-RUN COVERAGE IN GENERATED POPULATIONS

We first ask whether the generated population supplies correct answers beyond its best fixed member. All 35 admitted generated members and single-call have one scored execution on each of the same 386 analysis tasks: 13,896 of 13,896 cells are observed. Their union reaches 99.22% oracle coverage, versus 95.08% for the best fixed member, single-call. Every generated member is individually worse (69.69–92.49%). The resulting 4.15 pp headroom quantifies additional correct answers available to an oracle after execution.

The outcomes are diverse: 29 of 36 correctness vectors are unique, mean pairwise disagreement is 10.94% (maximum 33.16%), and the union repairs 84.21% of baseline errors. These quantities describe the population’s available correct answers. The repeat study evaluates a selected program panel against same-code controls on the same 386 tasks. Its  $D$  measures the panel’s held-out selection gain relative to clones, and its fresh baseline mean is 95.85%. The full-population headroom and panel estimates thus describe their respective populations and executions (Appendix A.1).

### 5.2 POSITIVE ORACLE ESTIMATES FROM IDENTICAL PROGRAMS

All nine clone slots implement the same program: their population-level stable complementarity is zero. On the common 386 tasks, their repeat-averaged oracle estimate is 2.16 pp, while the generated panel gives 2.33 pp (Table 2). Both estimates use identical task identities and complete observations. The control establishes that averaging three repeats does not make a positive oracle estimate identify specialization.

| Quantity                                                         | Generated + single-call | Same-code clones |
|------------------------------------------------------------------|-------------------------|------------------|
| Complete-task oracle estimate $\hat{H}$                          | 2.33 pp                 | 2.16 pp          |
| Complete tasks for repeat-based $G$                              | 386                     | 386              |
| Held-out-repeat mean gain $G$                                    | 0.10 pp                 | 0.35 pp          |
| Residual score-pattern correlation                               | 0.789                   | 0.006            |
| Paired clone-adjusted mean: $D = -0.26$ pp on 386 tasks          |                         |                  |
| Analysis-set completeness: 100% in both arms (10,422 cells each) |                         |                  |

Table 2: **Coverage, repeatability, and selection gain separate.** All quantities use the same 386 tasks. Both arms are fully observed on this analysis set.  $D$  is the paired difference of the unrounded cross-fitted gains; rounding may affect the displayed difference.

### 5.3 REPEATABLE SCORE PATTERNS AND HELD-OUT SELECTION GAIN

Generated programs do exhibit repeatable differences. An exploratory analysis removes additive member and task effects from the score-advantage matrix and correlates each held-out repeat with the other two. The mean residual correlation is 0.789 for generated programs and 0.006 for clones; their paired-bootstrap difference is 0.783 (95% interval  $[0.69, 0.87]$ ). These correlations describe repeatable *scored-outcome patterns*. To examine their direction, we count persistent losses and wins. All-three-repeat losses to single-call span 100 tasks (570 member–task pairs), while all-three-repeat wins occur on only one task, shared by all eight generated members. That apparent repair is sensitive to answer-format extraction (Appendix B.1). The observed asymmetry helps explain the coexistence of strong score repeatability and limited observed wins.

The held-out selection diagnostic asks the stronger question: do past outcomes identify a useful member for another execution? The clone-adjusted mean is  $D = -0.26$  pp, with 95% interval  $[-1.45, 0.95]$  pp and randomization  $p = 0.8127$ . All primary analysis tasks are fully observed. The decision nevertheless remains INSUFFICIENT EVIDENCE: the interval includes zero and the positive-support conditions are not met; the result does not establish equivalence. Excluding the three included tasks containing conflicting scores gives  $D = -0.61$  pp on 383 paired tasks and the same decision (Appendix B).

**Would the diagnostic detect real crossovers?** Known-truth calibration quantifies the diagnostic’s sensitivity. Across the tested 100-cell grid the worst observed null false-support rate is 2%, but a two-specialist setting with true  $H_{\text{stable}} = 17.5$  pp has 0% power at  $R = 3$ , about 0.7% at  $R = 5$ , and 100% at  $R = 10, 20$ . Smaller cyclic crossovers remain largely undetected even at  $R = 20$  (Appendix Table 7). These are setting-specific empirical results. Thus the data identify repeatable score patterns, while useful task-wise crossovers remain unresolved. Actual pre-execution selection can still be evaluated directly.

#### 5.4 PRE-EXECUTION SELECTION AND ORACLE REPLAY

**The frozen selector matches the development-selected baseline.** The final feature-policy choices, including its confidence-based fallback, are single-call on all 386 evaluation tasks. The result evaluates this complete decision rule; the reported choices do not separate the classifier’s initial preference from fallback decisions. Keeping those choices fixed and updating only the recorded outcomes gives 95.85% accuracy over three repeats, identical to the development-selected fixed member: 0.00 pp gain for the tested policy and features. Appendix D retains its construction and the same-set comparator results.

**At the full execution budget, the oracle coverages tie.** We compare development-ranked portfolios with uniformly selected same-code executions on the 386 tasks complete in both arms. At  $b = 3$ , the portfolio contains only single-call, so its +0.20 pp difference reflects execution variation between identical programs. At  $b = 9$  and 15, the differences are +0.08 pp and +0.02 pp. At  $b = 27$ , both arms reach 98.70% oracle coverage (Table 3).

| Harness executions per task $b$      | 3     | 9     | 15    | 27    |
|--------------------------------------|-------|-------|-------|-------|
| Panel members, including single-call | 1     | 3     | 5     | 9     |
| Panel oracle coverage (%)            | 98.45 | 98.70 | 98.70 | 98.70 |
| Clone oracle coverage (%)            | 98.24 | 98.63 | 98.69 | 98.70 |
| Panel minus clones (pp)              | +0.20 | +0.08 | +0.02 | +0.00 |

Differences are computed before rounding the displayed coverage values.

Table 3: **Full-budget oracle coverage is equal in the observed recovery snapshot.** Each included member contributes three repeats. Clone coverage is the exact mean over uniform subsets of its 27 recorded executions, paired on complete tasks. Both arms use post-execution correctness at matched harness execution counts.

These replay means describe post-execution oracle coverage at matched harness execution counts. Model-call counts and token costs vary by program. The tested pre-execution selector yields zero gain, while the full recorded execution budget provides equal oracle coverage for the two arms.

#### 5.5 SUPPORTING ANALYSIS OF EXECUTED BEHAVIOR

The MATH controls show that outcome differences need not require different programs. The supporting BIRD study asks the converse: do source differences produce distinct, useful execution? Here a mechanism is an implemented prompt transformation, tool use, or control-flow operation. The BIRD study examines execution directly in its own program populations, with task-specific judging and protocols.

**Source differences and observed outcomes.** All 12 discovery candidates pass a code-difference check, yet 21/66 generated-candidate pairs have identical correctness vectors (36/91 including baselines). Source similarity has essentially no rank association with correctness disagreement ( $\rho = -0.010$ ). The agreement on these 60 tasks motivates inspecting execution (Appendix Figure 2).

**Traces locate three distinct failures. T1: described but absent.** Six of twelve candidates claim retrieval or self-checking but make one model call and execute no artifact. This is a classification of all twelve candidates; the five-harness trace audit provides worked examples. **T2: present but un-triggered.** On all ten audited `react` tasks, the first SQL executes successfully, so the repair branch

never fires and final SQL equals single-call; execution success is not semantic correctness. **T3: executed but broken.** One candidate runs multi-turn repair but returns prose as final SQL, losing tasks that the baseline answers correctly. These failures require implementation checks, branch-exercising tasks, and valid output contracts, respectively. Appendix E retains worked traces, hand-written controls, and their development-exposure limitations.

The separate BIRD admission study also finds a mean headroom contrast of  $-0.16$  pp for forced/gated versus free/ungated generation. Its gate bundles structural exclusion with conformance verification (Appendix F). Together these results locate failures between source variation, mechanism implementation, activation, and valid output. Repeated task advantages and selection value measure the subsequent steps in the evidence chain.

## 6 DISCUSSION AND LIMITATIONS

**Implications for evaluation.** Our results separate three observations that can otherwise be treated as interchangeable evidence. Identical programs produce positive oracle headroom; generated programs produce repeatable score patterns; the frozen selector nevertheless realizes no gain, and full-budget oracle coverage ties. A claim of useful specialization must therefore connect repeatable *advantages*, task-wise crossover, and a usable selection rule. In the generated panel, persistent losses span 100 tasks, while persistent wins occur on one: the reproducible structure is largely a map of where programs lose, rather than where a selector can gain. If an advantage is attributed to a mechanism, traces must establish what ran and an intervention must test its contribution.

**Limitations.** The controlled MATH panel is a low-call draw under one solver on previously evaluated tasks. The revised held-out-repeat procedure and added repeatability analysis were developed after collection. The initial study ended under its stopping rule; the separately recorded recovery stage followed inspection of those earlier results, with programs, tasks, solver, and repeat count held fixed. The 386-task analysis set is fully observed and selected by completeness. The 14 excluded problems have higher observed difficulty and lower baseline accuracy (Appendix B); performance estimates therefore describe the selected cohort. Three repeats have low diagnostic power, and stable score patterns can include formatting effects from the fixed judge. The calibrated result remains INSUFFICIENT EVIDENCE. The zero routing gain concerns the tested selector and features. Replay standardizes harness execution counts; model-call counts and token costs vary by program. BIRD findings apply to its own judging and acquisition conditions.

## 7 CONCLUSION

For the tested MATH panel, more programs do not improve on more rolls at the full execution budget: both arms reach 98.70% oracle coverage after 27 executions, and the frozen feature selector gains 0.00 pp. Generated programs nevertheless exhibit strongly repeatable score patterns, with persistent losses spanning far more tasks than persistent wins. The same-code control, repeated executions, and frozen selector separate three distinct quantities: available correct answers, repeatable differences, and usable task advantages. Useful specialization requires the last of these: advantages that guide selection and improve on additional executions of a fixed program.

### ETHICS STATEMENT

All experiments use public BIRD and MATH-500 benchmark data and generated program artifacts; no human participants, private records, or personally identifiable information are collected. Model calls are reported as part of the experimental provenance, and the paper does not release credentials or provider responses that would expose private data. The main ethical risk is overstating what one-run oracle gains imply for deployed routing; we therefore report the unresolved selection and cost claims explicitly as INSUFFICIENT EVIDENCE.

### REPRODUCIBILITY STATEMENT

The manuscript specifies the populations, task splits, execution conditions, scoring rules, statistical procedures, and sensitivity analyses used for the reported results. The appendices provide addi-

tional protocol details, calibration results, and worked examples. The review materials present the methodological and empirical evidence through these descriptions, tables, and examples.

#### AI USE STATEMENT

This work is, by construction, an AI-assisted research project studying AI-generated artifacts, and we disclose the extent of that assistance precisely.

**Required disclosure.** We used generative AI tools for: (i) writing and modifying the experimental code (generation pipeline, evaluation collectors, statistical analysis scripts); (ii) running experimental workflows that made model API calls; (iii) analyzing experimental data and producing tables and figures; and (iv) drafting and editing manuscript text. The revised primary and factorial tables and result figure are generated from a shared, versioned reanalysis artifact. Historical exploratory analyses retain their original provenance and stated limitations. The post-review corrections are recorded separately from the original protocol. Verification records specify the scope of automated tests, development controls, and review for each artifact. Post-review revisions also used same-model subagent reviews for mechanical checks, supplemented by development controls and local runtime tests.

**Research ideas and direction.** The core research questions, the causal design decisions (estimand, contrasts, exclusion rules), the interpretation standards, and proposed go/no-go criteria were developed with AI assistance. The human authors set the overall research direction and requested the review-driven revision; AI tools also proposed analysis and experiment designs, identified counterexamples, and implemented revisions. Final scientific decisions and approval of the submission remain the authors’ responsibility.

**Responsibility.** We take responsibility for the final content of this work, including text, claims, and artifacts produced with the aid of generative AI tools. The authors have reviewed these disclosures and the final manuscript and are responsible for their accuracy.

#### REFERENCES

- Timothy A. Budd and Dana Angluin. Two notions of correctness and their relation to testing. *Acta Informatica*, 18(1):31–45, 1982. doi: 10.1007/bf00625279.
- Rich Caruana, Alexandru Niculescu-Mizil, Geoff Crew, and Alex Ksikes. Ensemble selection from libraries of models. In *Proceedings of the 21st International Conference on Machine Learning*, 2004. doi: 10.1145/1015330.1015432.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. doi: 10.48550/arXiv.2107.03374. URL <https://arxiv.org/abs/2107.03374>.
- Mengzhuo Chen, Junjie Wang, Zhe Liu, Yawen Wang, Haiming Zheng, and Qing Wang. From failed trajectories to reliable LLM agents: Diagnosing and repairing harness flaws. *arXiv preprint arXiv:2606.06324*, 2026a. doi: 10.48550/arXiv.2606.06324. URL <https://arxiv.org/abs/2606.06324>.
- Mingju Chen, Can Lv, Guibin Zhang, Heng Chang, and Shiji Zhou. HarnessForge: Joint harness and policy evolution for adaptive agent systems. *arXiv preprint arXiv:2606.01779*, 2026b. doi: 10.48550/arXiv.2606.01779. URL <https://arxiv.org/abs/2606.01779>.
- Yong-eun Cho. It’s not the capability: Harness sensitivity is non-monotone across LLM agent tiers. *arXiv preprint arXiv:2605.26731*, 2026. doi: 10.48550/arXiv.2605.26731. URL <https://arxiv.org/abs/2605.26731>.
- Batu El, Mert Yuksekgonul, and James Zou. Inefficiencies of meta agents for agent design. *arXiv preprint arXiv:2510.06711*, 2025. doi: 10.48550/arXiv.2510.06711. URL <https://arxiv.org/abs/2510.06711>.

- Tianyu Fan and Chao Huang. HELIX: Model-harness co-evolution for recursive self-improvement. *arXiv preprint arXiv:2608.13951*, 2026. doi: 10.48550/arXiv.2608.13951. URL <https://arxiv.org/abs/2608.13951>.
- Mitchell H. Gail and Richard Simon. Testing for qualitative interactions between treatment effects and patient subsets. *Biometrics*, 41(2):361–372, 1985. doi: 10.2307/2530862.
- Akshat Gupta, Jermaine Lei, Alexander Lu, Gopala Anumanchipalli, and Leshem Choshen. Automated discovery has no universally superior harness. *arXiv preprint arXiv:2607.18235*, 2026. doi: 10.48550/arXiv.2607.18235. URL <https://arxiv.org/abs/2607.18235>.
- Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*, 2024. doi: 10.48550/arXiv.2408.08435. URL <https://arxiv.org/abs/2408.08435>.
- Minho Kim. When should LLMs search? counterfactual supervision for search routing. *arXiv preprint arXiv:2607.05752*, 2026. doi: 10.48550/arXiv.2607.05752. URL <https://arxiv.org/abs/2607.05752>.
- Lars Kotthoff. Algorithm selection for combinatorial search problems: A survey. *AI Magazine*, 35(3):48–60, 2014. doi: 10.1609/aimag.v35i3.2460.
- Jacky Kwok, Shulu Li, Pranav Atreya, Yuejiang Liu, Yixing Jiang, Chelsea Finn, Marco Pavone, Ion Stoica, and Azalia Mirhoseini. LLM-as-a-verifier: A general-purpose verification framework. *arXiv preprint arXiv:2607.05391*, 2026. doi: 10.48550/arXiv.2607.05391. URL <https://arxiv.org/abs/2607.05391>.
- Jinyang Li, Binyuan Hui, Ge Qu, Jiayi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin Chang, Fei Huang, Reynold Cheng, and Yongbin Li. Can LLM already serve as a database interface? a BIG bench for large-scale database grounded text-to-SQLs. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, volume 36, pp. 42330–42357, 2023. doi: 10.52202/075280-1835. URL [https://proceedings.neurips.cc/paper\\_files/paper/2023/hash/83fc8fab1710363050bbd1d4b8cc0021-Abstract-Datasets\\_and\\_Benchmarks.html](https://proceedings.neurips.cc/paper_files/paper/2023/hash/83fc8fab1710363050bbd1d4b8cc0021-Abstract-Datasets_and_Benchmarks.html).
- Junyou Li, Qin Zhang, Yangbin Yu, Qiang Fu, and Deheng Ye. More agents is all you need. *arXiv preprint arXiv:2402.05120*, 2024. doi: 10.48550/arXiv.2402.05120. URL <https://arxiv.org/abs/2402.05120>.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*, 2023. doi: 10.48550/arXiv.2305.20050. URL <https://arxiv.org/abs/2305.20050>.
- Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, and Yu-Gang Jiang. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses. *arXiv preprint arXiv:2604.25850*, 2026. doi: 10.48550/arXiv.2604.25850. URL <https://arxiv.org/abs/2604.25850>.
- Xiaotian Luo, Dizhan Xue, Fengxingyu Wang, Chuanrui Hu, and Yafeng Deng. HarnessBank: Semantic gene-bank search with gated verification for agent-harness self-evolution. *arXiv preprint arXiv:2607.13683*, 2026. doi: 10.48550/arXiv.2607.13683. URL <https://arxiv.org/abs/2607.13683>.
- Atsuyuki Miyai, Kiyoharu Aizawa, and Toshihiko Yamasaki. Task-CoEvolve: Efficient harness optimization via adaptive validation task selection. *arXiv preprint arXiv:2608.20169*, 2026. doi: 10.48550/arXiv.2608.20169. URL <https://arxiv.org/abs/2608.20169>.

- Johannes Moll, Jean-Philippe Corbeil, Jiazhen Pan, Martin Hadamitzky, Daniel Rueckert, Lisa Adams, and Keno Bressem. GRASP: Gated regression-aware skill proposer for self-improving LLM agents. *arXiv preprint arXiv:2605.29668*, 2026. doi: 10.48550/arXiv.2605.29668. URL <https://arxiv.org/abs/2605.29668>.
- Jun Nie, Yonggang Zhang, Jun Song, Qianshu Cai, Dahai Yu, Yike Guo, Xinmei Tian, and Bo Han. TTHE: Test-time harness evolution. *arXiv preprint arXiv:2607.08124*, 2026. doi: 10.48550/arXiv.2607.08124. URL <https://arxiv.org/abs/2607.08124>.
- Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route llms with preference data. *arXiv preprint arXiv:2406.18665*, 2024. doi: 10.48550/arXiv.2406.18665. URL <https://arxiv.org/abs/2406.18665>.
- Shuyin Ouyang, Jie M. Zhang, Mark Harman, and Meng Wang. An empirical study of the non-determinism of ChatGPT in code generation. *arXiv preprint arXiv:2308.02828*, 2023. doi: 10.48550/arXiv.2308.02828. URL <https://arxiv.org/abs/2308.02828>.
- Mike Papadakis, Marinos Kintis, Jie Zhang, Yue Jia, Yves Le Traon, and Mark Harman. Mutation testing advances: an analysis and survey. In *Advances in Computers*, volume 112, pp. 275–378. Elsevier, 2019. doi: 10.1016/bs.adcom.2018.03.015.
- John R. Rice. The algorithm selection problem. In *Advances in Computers*, volume 15, pp. 65–118. Elsevier, 1976. doi: 10.1016/s0065-2458(08)60520-3.
- Ruhan Wang, Yucheng Shi, Zongxia Li, Zhongzhi Li, Yue Yu, Junyao Yang, Kishan Panaganti, Haitao Mi, Dongruo Zhou, and Leoweiliang. Harness handbook: Making evolving agent harnesses readable, navigable, and editable. *arXiv preprint arXiv:2607.13285*, 2026. doi: 10.48550/arXiv.2607.13285. URL <https://arxiv.org/abs/2607.13285>.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022. doi: 10.48550/arXiv.2203.11171. URL <https://arxiv.org/abs/2203.11171>.
- Jinghan Xu, Yikai Zhang, Aili Chen, Weiyuan Li, Jiaqing Liang, and Deqing Yang. Verify smarter, evolve further: Efficient harness evolution through behavior-aware verification. *arXiv preprint arXiv:2608.27311*, 2026. doi: 10.48550/arXiv.2608.27311. URL <https://arxiv.org/abs/2608.27311>.
- Lin Xu, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. SATzilla: Portfolio-based algorithm selection for SAT. *Journal of Artificial Intelligence Research*, 32:565–606, 2008. doi: 10.1613/jair.2490.
- Guibin Zhang, Leo Lu, Fangzhou Xie, Kang Zhu, Junhao Wang, Zhifei Xie, Zhaochen Yu, Zihang Liu, Zhongxiang Sun, Qiankun Li, Yue Liao, Heng Chang, Xiaobin Hu, Qibing Ren, Wangchunshu Zhou, Chuanrui Hu, Yafeng Deng, and Shuicheng Yan. JIT-Agent: Scaling harness intelligence via just-in-time harness evolution. *arXiv preprint arXiv:2608.25593*, 2026a. doi: 10.48550/arXiv.2608.25593. URL <https://arxiv.org/abs/2608.25593>.
- Hangfan Zhang, Shao Zhang, Kangcong Li, Chen Zhang, Yang Chen, Yiqun Zhang, Lei Bai, and Shuyue Hu. Self-harness: Harnesses that improve themselves. *arXiv preprint arXiv:2606.09498*, 2026b. doi: 10.48550/arXiv.2606.09498. URL <https://arxiv.org/abs/2606.09498>.
- Heng Zhou, Zelin Tan, Zhemeng Zhang, Yutao Fan, Yibing Lin, Li Kang, Xiufeng Song, Rui Li, Songtao Huang, Ao Yu, Yuchen Fan, Yanxu Chen, Kaixin Xu, Xiaohong Liu, Yiran Qin, Philip Torr, Chen Zhang, and Zhenfei Yin. Select-then-solve: Paradigm routing as inference-time optimization for LLM agents. *arXiv preprint arXiv:2604.06753*, 2026. doi: 10.48550/arXiv.2604.06753. URL <https://arxiv.org/abs/2604.06753>.

## APPENDIX GUIDE

The appendices document the studies supporting the current claims: study objects, repeat and selection methods, calibration, and BIRD evidence. Earlier generation pilots, superseded statistical procedures, gate-development iterations, and historical BIRD subset-selection analyses are outside this manuscript’s scope and remain in the project archive. Current analysis amendments and limitations are retained with the affected study.

**A** Harness Generation, Program Inventory, and Data Splits.

**B** Same-Code Controls and Repeated-Execution Protocol.

**C** Statistical Analysis, Calibration, and Sensitivity.

**D** Frozen Selector and Execution-Matched Replay.

**E** BIRD Execution Traces and Supporting Analyses.

**F** BIRD Admission-Policy Study.

## A HARNESS GENERATION, PROGRAM INVENTORY, AND DATA SPLITS

We first document the MATH source population, task splits, and frozen panel, then list model versions. The BIRD discovery and admission protocols are in Appendices E and F.

### A.1 MATH POPULATION, COMMON ANALYSIS SET, AND COVERAGE

The single-run study supplies the initial coverage evidence and the programs for the repeat panel. It carries the BIRD admission study’s free, ungated generation procedure into competition mathematics, with a domain-specific answer comparator.

**Protocol and analysis set.** The builder task can be summarized as follows (a structured summary, not a verbatim prompt): write a Python harness implementing `solve(question)`; use `self.llm` to improve on a single greedy call; choose the prompting, decoding, sampling, and control flow; return the final mathematical answer with the requested ##### marker. Gold answers belong to the external judge. No slot is assigned a mechanism and no cross-program diversity target is imposed. Repeated attempts use the same instruction, without earlier candidate code or performance feedback. Admission checks interface validity and nonempty output, not an accuracy improvement.

Six builders receive eight slots each under one seed, with at most three raw attempts per slot. The neutral-validity gate checks parsing, import, interface, and nonempty answers on two development questions. The frozen target is GLM-5.3-Flash, with member-specific decoding and temperature 0 for single-call. The original task split precedes generation and reserves 100 development problems. The final evaluation analysis uses the same 386 fully observed tasks for the single-run population, both repeat arms, and frozen-policy scoring. This task set is determined after collection by completeness; 14 incomplete problems are excluded (Appendix B). The development tasks and frozen selector remain unchanged.

The judge compares numeric values when both sides parse as numbers (including fractions and decimals), and normalized LaTeX otherwise. Normalization strips units, base subscripts, and degree symbols, preserves text-command content, and maps matrices to tuples. The requested final-answer marker is #####. Formatting each gold answer with this marker gives 386/386 correct self-comparisons, verifying format consistency on these gold answers. The observed single-run baseline accuracy is 95.08% on the analysis set.

The validity gate admits 35/48 slots. The two reasoning-heavy builders, GLM-5.3 and Qwen3.8-Max, exhaust the 32k-token generation budget before emitting code in most slots. This under-represents those builder styles under the fixed generation budget. The exploratory study evaluates free generation with neutral-validity admission under one seed.

**Results on the common complete set.** All 36 members, including single-call, have one recorded score on each of the 386 tasks (13,896/13,896 cells). Mean pairwise disagreement is 10.94% (maximum 33.16%). There are 29/36 unique member outcome vectors (28/35 generated vectors), and 2.22% of member pairs have identical outcomes. The union of generated candidates repairs 84.21% of baseline errors and fails on 0.82% of baseline-correct problems. The latter measures the fraction of baseline-correct tasks on which all generated members fail. Oracle coverage is 99.22%, versus 95.08% for the best fixed member, single-call, giving 4.15 percentage points of headroom. Generated-member accuracies range from 69.69% to 92.49%.

The fresh repeat panel uses eight generated members plus single-call and nine same-code slots on these identical task identities. Its clone-adjusted diagnostic measures held-out selection gain for this panel; the one-run headroom characterizes the full generated population. Replaying the archived judge reproduces all 13,896 included scores; the versioned endpoint-sensitive judge produces identical scores on this subset.

**Scope.** The MATH findings characterize exploratory generation under one seed and one target on a completeness-selected evaluation set. The BIRD studies characterize SQL execution and admission under their respective protocols.

## A.2 SELECTED MATH PROGRAM INVENTORY

Table 4 describes the nine frozen members selected for the repeat study. H1–H8 are neutral labels for generated members, all constructed under seed 0. The listed operations describe what each program implements, not whether every branch fires or improves correctness. The mean-call eligibility ceiling is not a per-execution maximum, and one logical call may request several samples.

| Frozen member | Source-defined execution path                                                                                                              |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| single-call   | One temperature-0 solution with the final-answer format.                                                                                   |
| H1 (DeepSeek) | Three temperature-0.4 solutions; normalized-answer vote; a greedy fall-back when no answer or a tie remains.                               |
| H2 (ERNIE)    | Draft a solution, then ask the solver to verify/correct the extracted answer; return the checked answer.                                   |
| H3 (GLM)      | Up to five differently prompted temperature-0 solutions; stop when three agree; adjudicate unresolved disagreement, with format fallbacks. |
| H4 (Kimi)     | Request five temperature-0.7 samples in one call; extract answers and vote.                                                                |
| H5 (Kimi)     | Initial solution, verification, and an alternative solution; vote, with an additional resolution call if no majority emerges.              |
| H6 (MiniMax)  | Draft, critique, and an additional tie-breaking call on disagreement; parse failures use an earlier available answer.                      |
| H7 (MiniMax)  | Request five temperature-0.7 samples in one call; normalize and vote, resolving ties by first occurrence.                                  |
| H8 (Qwen)     | Draft plus an independent solve-and-audit call; compare answers and request adjudication on conflict, with a format-repair path.           |

Table 4: **What was selected for the repeat study.** All calls use the same frozen solver; verification and adjudication are solver prompts, not access to benchmark labels. Six seats are builder-stratified; H5 (Kimi) and H7 (MiniMax) are the two at-large seats.

## A.3 MODELS AND VERSIONS

Table 5 lists the models used in the reported benchmark studies with their exact version strings as sent to the API. All models were accessed through a single commercial OpenAI-compatible API aggregator (the same provider for every role). The frozen target model identity is fixed; decoding follows each harness, with temperature 0 for the single-call baseline. Phase-II harness generation uses temperature 0.7 with a 16,384-token budget. Phase-I generation conditions and their evidence gaps are described in Appendix E.1. Phase I used a proxy execution judge; Phase II uses the official BIRD scorer (30 s timeout), with the legacy judge as a secondary audit. The MATH-500 probe uses the answer comparator in Appendix A.1; no LLM judge produces reported accuracy.

| Role                           | Model (exact version string)                                                           | Used in     |
|--------------------------------|----------------------------------------------------------------------------------------|-------------|
| Frozen target solver           | GLM-5.3-Flash (baseline temperature 0)                                                 | MATH / BIRD |
| BIRD discovery builder         | GLM-5.3-Flash                                                                          | App. E.1    |
| MATH / BIRD admission builders | GLM-5.3, Qwen3.8-Max, DeepSeek-V4-Pro, Kimi-K3, MiniMax-M3, ERNIE-5.0-Thinking-Preview | MATH / BIRD |
| Judge                          | proxy / official BIRD execution scorer; MATH answer comparator                         | see text    |

Table 5: Model inventory for the reported benchmark studies. The target is held frozen within each phase; Phase-II builders come from six independent labs.

## B SAME-CODE CONTROLS AND REPEATED-EXECUTION PROTOCOL

All current MATH primary estimates use one 386-task, fully observed analysis set. This section records how it was obtained and documents score completeness, provenance, and sample selection.

**Study objects, capture, and inclusion rule.** The program panel, solver, judge, development split, and three repeat identities remain fixed. A salted-hash draw selects one low-call member per builder and two at-large seats, then adds single-call; the comparator has nine identical baseline slots. The score snapshot captured on September 26, 2026 at 05:15 UTC combines generated-arm ledgers through continuation 10 and clone-arm ledgers through continuation 20. This snapshot fixes the analysis inputs. We retain exactly the tasks with a scored outcome for every required member and repeat in both arms and every member in the single-run study, accepting either recorded binary score at each required cell. The resulting 386 task identities are frozen for every primary comparison. This inclusion rule is applied after outcome inspection and is disclosed as a reanalysis.

**Complete analysis set and excluded observations.** The single-run matrix contains  $36 \times 386 = 13,896$  observed cells. Each repeat arm contains  $9 \times 3 \times 386 = 10,422$  observed cells. All three matrices therefore have 100% scored-observation completeness on the analysis set. The same identities are used for policy accuracy and oracle replay. Score completeness and acquisition accounting are recorded separately.

The fixed inclusion rule excludes 14 problems with 61 unscored generated-arm cells; the clone records on these excluded problems are complete. These problems are excluded from the present analysis, and their unknown scores remain unknown in the retained source records. The original excluded-task fraction is 3.50%, below the frozen 10% gate, assessed on the pre-subsetting acquisition records. By builder, unscored cells are DeepSeek 5, ERNIE 4, GLM 9, Kimi 30, MiniMax 9, and Qwen 4. The baseline is complete. Source records and incurred acquisition costs remain preserved, including those outside the analysis set.

| Observed characteristic           | Included (386) | Excluded (14) |
|-----------------------------------|----------------|---------------|
| Baseline repeat-mean accuracy (%) | 95.85          | 71.43         |
| Mean benchmark difficulty level   | 3.47           | 4.29          |
| Level-5 tasks (%)                 | 26.42          | 64.29         |
| Mean question length (characters) | 198.75         | 284.71        |

Table 6: **Observed characteristics of included and excluded tasks.** Excluded tasks have higher benchmark difficulty, longer questions, and lower baseline accuracy.

**Identity, precedence, and conflicts.** Continuation manifests are checked on 13 condition fields, including solver, cache, judge, protocol, repeats, program identities, task/split hashes, panel draw, and schedule. The original snapshot hashes are verified and the new score matrices and task lists are hash-bound. For each member-task-repeat identity, a completed score replaces an unknown; the first completed score takes precedence when later records differ. Five generated-arm cell identities have conflicting scores, three of which lie on three included tasks; clones have none. Collector and concurrency changes remain recorded as operational deviations.

**Current estimates and sensitivity.** The common-set plugins are 2.33 pp (generated) and 2.16 pp (clone). The paired diagnostic is  $D = -0.26$  pp, with 95% interval  $[-1.45, 0.95]$  pp and INSUFFICIENT EVIDENCE. A conflict-exclusion sensitivity removes the three affected included tasks from both arms and recomputes all choices on the remaining 383 fully observed tasks:  $D = -0.61$  pp, interval  $[-1.96, 0.64]$  pp, with the same state. This explicitly labeled sensitivity subset is separate from the 386-task primary analysis. Both estimates are computed directly from recorded binary scores. Versioned artifacts preserve earlier estimates and acquisition snapshots.

### B.1 EXPLORATORY REPEATABILITY AND SCORE SENSITIVITY

For each arm, let  $A_r(h, x) = Y(h, x, r) - Y(h_0, x, r)$  for non-reference members, with  $h_0$  =single-call or clone-cl. On the common complete tasks, subtract the row and column means and add the grand mean separately within each repeat. For each held-out repeat, correlate the flattened residual matrix with the residual of the other two repeats' mean; report the mean of the three correlations. A paired task bootstrap with 2,000 draws, resampling the same tasks in both arms, supplies intervals. Generated correlation is 0.789  $([0.75, 0.83])$ , clone correlation 0.006  $([-0.07, 0.08])$ , and their difference 0.783  $([0.69, 0.87])$ . This post-collection diagnostic measures residual score-pattern repeatability after removing additive effects.

The recorded-score asymmetry is substantial: generated members lose to single-call in all three repeats on 570 member-task pairs spanning 100 distinct tasks. All-three-repeat wins comprise eight pairs but only one task, shared by all eight generated programs. These counts summarize the observed three-repeat score patterns. In that single task, the gold answer is 12; all baseline outputs conclude ##### 12th grade, whereas a generated member returns 12th grade. The fixed extractor preserves the complete ##### payload and scores the former zero, but falls back to numeric extraction and scores the latter one. The score difference arises from the extractor's format-dependent branches. We retain the recorded scores and interpret this example as a format-sensitive repeated repair.

## C STATISTICAL ANALYSIS, CALIBRATION, AND SENSITIVITY

This section specifies the held-out-repeat diagnostic used in the main text. Empirical estimates use the common 386 complete MATH tasks; synthetic calibration uses the separately specified sample sizes.

### C.1 HELD-OUT-REPEAT STATISTIC AND DECISION RULE

In each of three folds, two discovery repeats select a member for each task and a single best fixed member by mean discovery accuracy. The third repeat scores the task-wise mapping against that fixed member. Averaging task-level gains over folds gives  $G$ ; the paired difference between arms is  $D = G_{\text{real}} - G_{\text{clone}}$  (§4.2). The repeat-averaged oracle plug-in  $\hat{H}$  is descriptive and does not determine the decision.

Discovery scores lie on  $\{0, \frac{1}{2}, 1\}$ , so ties are common. Selection uses iid  $U(0, 10^{-9})$  jitter to break member ties symmetrically, averaging ten frozen tie realizations for the empirical report. Each calibration dataset uses one tie realization. For the empirical randomization reference, member labels are permuted independently within each task and each repeat in each arm. Both selections, both gains, and  $D$  are recomputed for each of 10,000 draws, with the +1 Monte Carlo correction. A paired task bootstrap uses 2,000 draws with a fixed random seed and the same resampled task identities in both arms to obtain the 95% interval.

Positive support requires all of the following: at most 10% incomplete tasks in either arm or their intersection, assessed before complete-case subsetting; at least 30 paired tasks; randomization  $p < 0.05$ ; a positive bootstrap lower bound for  $D$ ;  $D > 1.0$  pp; and a positive lower bound for  $G_{\text{real}}$ . The current comparison is INSUFFICIENT EVIDENCE; it does not establish equivalence. Execution-condition and program-identity checks require matched, independently acquired repeats. Identical outcome matrices alone do not establish duplicated acquisition: distinct execution provenance must be verified.

**Analysis amendments.** The acquisition protocol preceded collection; the current v3 diagnostic and completeness-selected analysis were determined afterward. The original stopping rule applied to the initial collection. Recovery and the common-set reanalysis are separately recorded amendments, with the present estimates tied to the September 26 snapshot. Appendix B gives the inclusion rule, excluded-task characteristics, score conflicts, and conflict-exclusion sensitivity. Earlier diagnostic versions are superseded and do not supply the current estimates.

## C.2 POPULATION TARGET AND LIMITS OF THE DIAGNOSTIC

For member success probabilities  $q_h(x)$ , stable complementarity is

$$H_{\text{stable}} = \mathbb{E}_x \max_h q_h(x) - \max_h \mathbb{E}_x q_h(x).$$

It is zero when some fixed member is expectation-best on almost every task. A selector’s population gain  $\theta^*$  relative to the *population-best fixed* member satisfies  $\theta^* \leq H_{\text{stable}}$ , because its success probability on each task is at most  $\max_h q_h(x)$ .

The empirical  $G$  instead compares with a fixed member selected from finite discovery data, which need not identify the population-best member. Clone subtraction supplies an observed sampling reference; it does not analytically remove all finite-sample selection bias. The reported interval for  $D$  is therefore not an interval for  $H_{\text{stable}}$ . A near-zero  $D$  does not bound stable complementarity near zero.

The randomization reference is exact at the equal-ability boundary under member-label exchangeability. For the wider global-dominance null, error control is checked empirically on the calibration grid, not established by a general finite-sample theorem.

**Finite-repeat oracle estimates.** For repeat means  $\hat{q}_h(x)$ , Jensen’s inequality gives  $\mathbb{E} \max_h \hat{q}_h(x) \geq \max_h \mathbb{E} \hat{q}_h(x)$  on each task. Headroom subtracts a second maximum, so this inequality does not imply upward bias of the difference in general. In a known-truth equal-ability simulation with  $q = 0.9$ ,  $M = 9$ ,  $T = 400$ , and  $R = 3$ , the mean plug-in is 8.69 pp over 300 datasets despite  $H_{\text{stable}} = 0$ . The empirical same-code arm likewise gives 2.16 pp. These examples show that positive plug-in headroom is insufficient evidence of specialization.

## C.3 KNOWN-TRUTH CALIBRATION AND STATISTICAL POWER

Binary tensors are sampled from specified member–task probability matrices. Null settings include equal ability, shared task difficulty, global dominance, the 0.5/0.9 dominance configuration, heterogeneous dominance, and matched sticky within-task latent states. Alternatives include crossovers over latent task types and difficulty-related partial crossover. The grid spans  $M \in \{3, 9, 20\}$ ,  $T \in \{100, 400, 1000\}$ ,  $R \in \{3, 5, 10, 20\}$ , complete data, 5% MCAR, and 3% hardness-related missingness.

Table 7 retains the  $M = 9, T = 400$  settings, with 300 Monte Carlo datasets per cell (150 in sweeps) and 300 randomizations per dataset. It compares plug-in support with the current clone-calibrated rule. Across the tested 100-cell grid, the worst observed null false-support rate for that rule is 2%. A two-specialist alternative with true  $H_{\text{stable}} = 17.5$  pp has 0% power at  $R = 3$ , about 0.7% at  $R = 5$ , and 100% at  $R = 10, 20$ . Smaller cyclic crossovers remain largely undetected even at  $R = 20$ . These setting-specific rates do not establish a general size or power guarantee; three repeats leave stable complementarity unresolved in the current study.

| regime                              | $R=3$ |    | $R=10$ |     | $R=20$ |     | abst. |
|-------------------------------------|-------|----|--------|-----|--------|-----|-------|
|                                     | plug  | v3 | plug   | v3  | plug   | v3  | $R=3$ |
| equal $q=0.9^\dagger$               | 100   | 0  | 100    | 0   | 100    | 1   | 100   |
| equal + difficulty <sup>†</sup>     | 100   | 0  | 100    | 1   | 100    | 1   | 100   |
| 0.5/0.9 dominance <sup>†</sup>      | 100   | 0  | 100    | 0   | 13     | 0   | 100   |
| dominance <sup>†</sup>              | 100   | 0  | 100    | 0   | 100    | 0   | 100   |
| cross $H \approx 0.9$ pp            | 100   | 1  | 100    | 0   | 100    | 0   | 99    |
| cross $H \approx 1.8$ pp            | 100   | 1  | 100    | 1   | 100    | 0   | 99    |
| cross $H \approx 4.2$ pp            | 100   | 0  | 100    | 0   | 100    | 0   | 100   |
| two strong specialists, $H=17.5$ pp | 100   | 0  | 100    | 100 | 100    | 100 | 100   |

<sup>†</sup>Null ( $H=0$ ): false-support %, nominal 5%; else power %.  $M=9$ ,  $T=400$ ,  $B=300$  at  $R=3$  (150 sweeps), 300 randomizations.

Appendix C describes the  $M$ ,  $T$ ,  $R$  and missingness settings and summarizes the wider calibration grid.

**Table 7: Three repeats can miss even a strong positive case.** At  $M = 9$ ,  $T = 400$ , null rows report false-support % and alternative rows report power %; the final column gives  $R = 3$  abstention. The clone-calibrated rule (v3) rarely supports nulls, but has little sensitivity to the tested small and moderate crossovers.

## D FROZEN SELECTOR AND EXECUTION-MATCHED REPLAY

The selector is fitted on 100 development tasks and evaluated on the same 386 fully observed analysis tasks as every other current MATH comparison. We apply the recorded selector choices and development ranking to this common set.

### D.1 FROZEN FEATURE SELECTOR

The reported selector  $\pi_Z$  combines five multinomial-logistic fold models fitted using the 100 development tasks. Its inputs are TF-IDF question features (at most 300) and per-member development accuracies computed within the respective training folds. All repeats of a task stay together in the five-fold task partition. The score summary for member  $h$  on development task  $x$  is the available-repeat mean,

$$\bar{Y}_h(x) = \frac{1}{|\mathcal{R}_{hx}|} \sum_{r \in \mathcal{R}_{hx}} Y(h, x, r),$$

where  $\mathcal{R}_{hx}$  contains its observed development repeats. The five fitted models are combined by fold-model averaging. If  $p_k(h | x)$  is the probability assigned by fold model  $k$ , the inference rule uses  $\bar{p}(h | x) = \frac{1}{5} \sum_{k=1}^5 p_k(h | x)$ . Let  $h_{(1)}$  and  $h_{(2)}$  be the members with the two highest averaged probabilities, breaking ties by member ID. The final frozen choice is

$$\pi_Z(x) = \begin{cases} \text{single-call,} & \bar{p}(h_{(1)} | x) - \bar{p}(h_{(2)} | x) < 0.15, \\ h_{(1)}, & \text{otherwise.} \end{cases}$$

Thus the five-fold step supplies the averaged prediction used at inference, and the threshold is applied after averaging. The policy table evaluates these final choices, including the baseline fallback. It reports repeat-mean accuracy, with harness executions as the unit of evaluation; a selected harness can make multiple model calls.

Comparators are single-call, the development-selected fixed member, one randomly drawn fixed member (drawn once with a fixed seed), and the development-selected member with a baseline fallback on missing cells. The random comparator uses the same member for every task.

| Policy                          | Eval acc. (%) | $S$ vs dev-fixed (pp) [95% CI] |
|---------------------------------|---------------|--------------------------------|
| single-call (fixed)             | 95.85         | 0.00 [0.00, 0.00]              |
| dev-fixed                       | 95.85         | —                              |
| One randomly drawn fixed member | 86.01         | -9.84 [-13.04, -6.91]          |
| Dev-fixed + baseline fallback   | 95.85         | 0.00 [0.00, 0.00]              |
| $\pi_Z$ (frozen selector)       | 95.85         | 0.00 [0.00, 0.00]              |

Table 8: **Frozen comparators on the common 386-task MATH analysis set.** Every selected member has all three scores on every task. Accuracy is repeat-mean correctness; paired differences are against dev-fixed with 2,000 task-bootstrap draws. The feature policy, dev-fixed, and fallback policy all retain their recorded single-call choices. The randomly drawn fixed member is `gsm_kimi_s0_g3`. Recorded policies are evaluated on fresh executions of these tasks.

## D.2 SELECTION GAIN AND THE SCOPE OF BUDGET UTILITY

For the implemented selector,  $Z(x)$  consists only of question features and training-fold member accuracies. Its gain over the development-selected fixed member is

$$S(\pi_Z) = \mathbb{E}_x [\bar{Y}_{\pi_Z(x)}(x) - \bar{Y}_{h_{\text{dev-fixed}}}(x)],$$

where  $\bar{Y}_h(x)$  now averages the three evaluation repeats. We use  $S$  for policy gain and reserve  $D$  for the real-minus-clone repeat diagnostic. Mechanism labels are not inputs to this feature selector.

The replay below measures oracle coverage at matched *harness execution* counts. Its budget  $b$  counts harness executions, each of which may contain multiple model calls.

## D.3 EXECUTION-MATCHED REPLAY AND COLLECTION ACCOUNTING

Equal-slot collection executes nine members/slots for three repeats in each arm, with call counts and token use determined by each program. The current execution-matched replay (§5.4) instead uses the 386 common complete tasks and the unchanged development ranking. At budgets  $b \in \{3, 9, 15, 27\}$  it includes all three repeats of the first  $b/3$  ranked members. For a task with  $f$  incorrect clone executions out of 27, the expected coverage of a uniformly selected size- $b$  subset is  $1 - \binom{f}{b} / \binom{27}{b}$ , with  $\binom{f}{b} = 0$  for  $b > f$ . We compute the exact uniform-subset expectation. Table 3 reports the paired means. Both arms use post-execution oracle correctness, yielding descriptive coverage estimates at matched harness execution counts.

Acquisition accounting remains separate from these outcome tables. The raw ledgers retain all recorded attempts and known usage, including excluded tasks and recovery. These ledgers record incurred acquisition costs over their full scope, while the outcome tables describe the common complete task set. Generated members can make multiple calls; resource use is recorded at the program and acquisition-record level.

## E BIRD EXECUTION TRACES AND SUPPORTING ANALYSES

This appendix expands the supporting execution diagnosis. In the historical records, Phase I denotes BIRD discovery and Phase II the separate BIRD admission study; neither explains the MATH results causally.

### E.1 DISCOVERY POPULATIONS AND CONDITIONS

**Population and judging.** The discovery comparison in §5.5 evaluates twelve TTHe-proposer candidates from a GLM-5.3-Flash builder, plus single-call and `react`, on 60 tasks. Admission checks import and smoke execution; it does not validate the claimed mechanism. The baseline flipped three of the 60 items between discovery runs, so their baseline-error sets differ slightly. Discovery used a set-of-rows proxy judge. On the two matrices with retained final SQL, its agreement with the official BIRD scorer is 0.97–0.99, with no proxy-correct/official-incorrect cases.

**Development exposure and record limits.** The hand-written controls were designed after inspecting discovery traces; 24 of their 151 evaluation items overlapped protocol-development measurements. Their headroom is descriptive, not a blind estimate of stable complementarity. Early in-session generation records do not retain the generation tool and per-attempt logs needed to audit equal generation budgets. These limitations apply to the discovery evidence retained here.

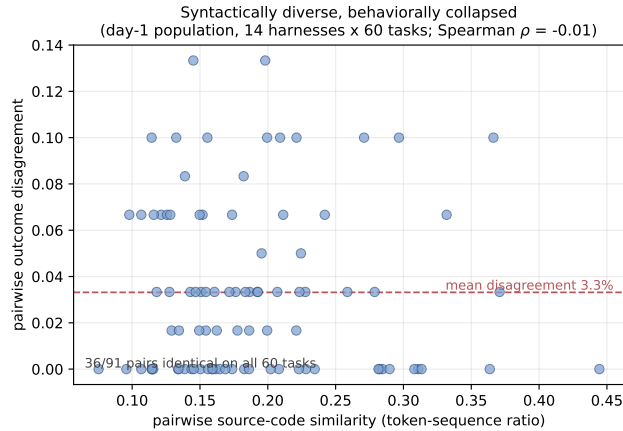


Figure 2: **Source differences need not change observed correctness.** BIRD discovery: 14 harnesses, 60 tasks, one execution each. Across 91 pairs including baselines, token-sequence source similarity has  $\rho = -0.010$  rank association with correctness disagreement; 36/91 pairs have identical correctness vectors. This is an outcome comparison on the observed tasks, not a test of program equivalence.

## E.2 DISCOVERY MEASUREMENTS AND HAND-WRITTEN CONTROLS

Every BIRD discovery candidate passes a code-difference check. Yet 21/66 generated-candidate pairs have identical correctness vectors (36/91 including baselines), and source similarity has essentially no rank association with correctness disagreement across the 91 pairs ( $\rho = -0.010$ ; Figure 2). Mean disagreement is 3.3% overall (3.8% among generated candidates), union repair of single-call errors is 7.7%, and one-run headroom is 3.3 pp. Agreement on these 60 tasks does not establish general program equivalence, but it motivates examining what the programs actually do.

## E.3 TRACES DISTINGUISH ABSENCE, NON-ACTIVATION, AND FAILURE

**T1: described but absent.** Six of twelve candidates claim retrieval or self-checking in docstrings but make a single model call and execute no artifact. The named mechanism is not implemented. **T2: present but untriggered.** In all ten audited `react` tasks, the first SQL executes successfully, so an existing execute-feedback-repair branch never fires; final SQL equals single-call. Successful execution is not semantic correctness, and these runs do not test repair under failure. **T3: executed but broken.** One of twelve candidates performs multi-turn execution and repair but returns prose as final SQL, losing tasks that single-call answers correctly.

These observations locate different failures rather than assigning one diversity score. T1 requires checking implementation; T2 requires tasks that exercise the branch; T3 requires a valid output contract. They do not establish that activation causes improved accuracy. Four hand-written controls with distinct paths produce 6.67 pp one-run headroom on 60 tasks and 7.28 pp on 151 items, showing that the apparatus can record different repairs. But they were written after trace inspection and lack matched repeats and clone controls. Appendix E.1 discloses the legacy proxy and 24/151 items overlapping protocol development; Appendix E.4 gives worked traces.

#### E.4 WORKED EXAMPLES FROM THE TRACE AUDIT

The T1 count above classifies all twelve discovery candidates. The separate trace audit covers five harnesses: `single-call`, `react`, the single-call candidate, the timeout-retry candidate, and the multi-turn repair candidate. It illustrates execution paths rather than providing the denominator for the six-of-twelve T1 count. This section walks one task through those paths. Outputs are quoted from this audit (5 harnesses  $\times$  10 tasks); its logger caps stored SQL excerpts at 120 characters, so truncations are marked *[trunc.]*.

**The task.** BIRD dev-set question 352 (`card_games` database; question text verbatim, the typo is the dataset’s):

```
Calculate the percentage of the cards availabe in Chinese
Simplified.
```

Its BIRD hint:

```
'Chinese Simplified' is the language; percentage =
Divide(Sum(id where language = 'Chinese Simplified'),
Count(id)) *100
```

Gold SQL (executing the two on the released database yields 8.7734 and 8.7728 respectively — the official scorer separates them):

```
SELECT CAST(SUM(CASE WHEN T2.language = 'Chinese Simplified'
THEN 1 ELSE 0 END) AS REAL) * 100 / COUNT(T1.id) FROM cards
AS T1 INNER JOIN foreign_data AS T2 ON T1.uuid = T2.uuid
```

The single-call harness answers:

```
SELECT SUM(CASE WHEN language = 'Chinese Simplified'
THEN 1 ELSE 0 END) * 100.0 / COUNT(id) AS percentage FROM
foreign_data;
```

marked wrong by the official scorer: it counts directly over `foreign_data`, whereas the gold query counts after joining to `cards`; the hint does not spell out that join.

**react (T2: mechanism present but untriggered).** `react`’s control flow does execute SQL and retries on database errors. But its first call is byte-identical to `single-call`’s prompt, and here the generated SQL executes without error, so the error-triggered retry never fires and the final SQL equals `single-call`’s. Across all 10 audited tasks the pattern held: SQL executions present, final SQL byte-identical to `single-call`’s on 10/10 — the feedback loop is real code whose repair branch did not activate on these tasks. Controlled failure injection is needed to test whether feedback changes the output.

**Single-call candidate (T1: mechanism described, not executed).** This candidate’s docstring claims a “conservative repair”: cap the solver’s maximum output tokens and demand an answer-only block. Its behavior is structurally `single-call`’s: one LLM call followed by SQL extraction. The token cap exists only as an environment default (2,048 against the runtime’s 32,000) — no harness control flow implements it, and it never binds on these tasks because a SQL answer is orders of magnitude below the cap. On the worked task it issues exactly one call, executes nothing, and returns the same wrong query as `single-call` modulo whitespace (counting over `foreign_data` without the join). Across the 10 audited tasks it records zero SQL executions on all 10, and its final SQL is byte-identical to `single-call`’s on 6 (the remaining differences trace to the reworded system prompt, with no implemented retrieval, self-check, or repair operation). Prompt rewriting is an implemented transformation; it does not implement the additional mechanism claimed in the docstring. The most common Phase-I failure is thus a claimed operation absent from the executed program.

**Timeout-retry candidate (code-level diversity, behavioral identity).** This candidate is a genuinely different program: it wraps the single coder call in a daemon-thread watchdog and fires one

hedged duplicate re-ask if the call stalls past 40 s. The mechanism exists in the control flow — but its trigger (a stalled API call) never occurred in the audit: zero SQL executions, and the final SQL equaled single-call’s on all 10 audited tasks. Code difference without behavioral difference survives even when the added mechanism is real: it simply never fires.

**Multi-turn repair candidate (T3: a broken multi-turn path).** The one audited candidate implementing genuine execute-inspect-repair returns the model’s *prose* as final SQL. On a task single-call answered correctly (“State the alternative languages available for card named Annul numbered 29.”), its logged final SQL begins, verbatim:

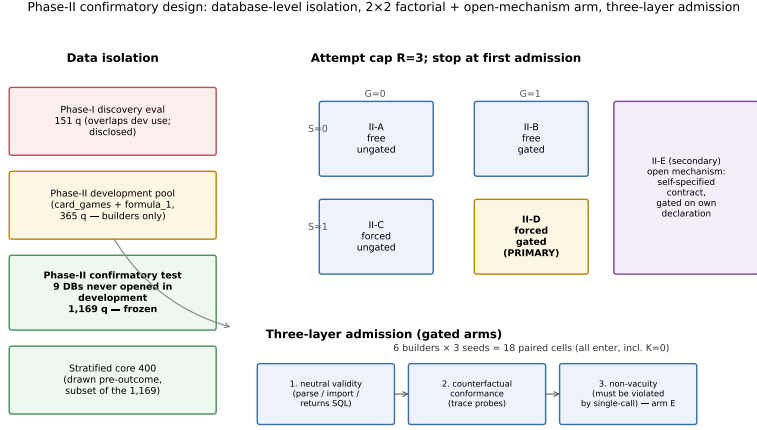
```
with "only a single read-only SELECT is allowed". The
previous "query" was actually just prose text (the model
wrote rea[trunc.]
```

The repair re-prompt invites the failure: asked to fix a query that “was actually just prose”, the frozen target responds with more prose reasoning, no SQL block is found, and the reasoning text is submitted as the query. The item flips from single-call-correct to harness-wrong.

## F BIRD ADMISSION-POLICY STUDY

The confirmatory study uses a  $2 \times 2$  factorial design with six builders and three seeds; open-mechanism arm II-E is development-only. Each slot allows at most  $R = 3$  attempts, stopping at first admission without top-up. Two development databases are excluded from the evaluation pool. The evaluation set contains 1,169 questions from nine databases and a pre-drawn 400-item core. An 18-item pipeline smoke test is a disclosed exception to development isolation. The official BIRD judge enforces a 30 s timeout. Responses are cached by model identity with cross-process locking; a cache-concurrency defect found during collection was corrected.

The original primary contrast was D—A on paired builder–seed cells with hierarchical bootstrap and permutation inference, with Holm-corrected factorial secondary analyses on the common core and K-matched sensitivity analyses. Instrument-calibration fixes preceded confirmatory outcomes. Other deviations included operational access handling and quarantine of inconsistent observations. The corrected analysis uses shared baseline records and database/task resampling; these revisions followed the original analysis specification. Overlapping acquisition records are deduplicated on canonical identity using the recorded first-write-wins rule; verdict conflicts remain in the source audit. These tables use that deduplicated view. The implemented gate can structurally reject permitted prompt-only transformations despite neutral validity, and realized generation costs differ across arms.



**II-E: development-only; excluded from confirmatory comparisons.**

**Figure 3: BIRD admission design.** Left: database-level contamination isolation. Middle: the 2x2 confirmatory factorial with a common attempt cap; the additional open-mechanism arm II-E belongs only to development. Right: the three-layer admission pipeline.

#### F.1 DESIGN AND PRIMARY CONTRASTS

A separate confirmatory BIRD study crosses six independent builders with three seeds under strategy forcing (a named mechanism from a fixed set versus builder-chosen strategy) and conformance gating, giving four arms: II-A (free, ungated), II-B (free, gated), II-C (forced, ungated), and II-D (forced, gated). “Free” means that no specific strategy is assigned; the builder still declares repair, vote, twostage, or plain, as detailed in Appendix F.4. The evaluation covers 1,169 questions from nine databases never opened during development (one disclosed 18-item smoke-test exception), with the official BIRD execution scorer and a pre-outcome-frozen analysis later re-run as a versioned corrected reanalysis. The implemented gate bundles nontrivial-mechanism structural screening with conformance verification — permitted prompt-only transformations can fail structural screening despite passing neutral validity checks — so the treatment does not isolate verification causally.

| Metric                       | II-A           | II-D  |
|------------------------------|----------------|-------|
| Candidate count              | 5.78           | 3.83  |
| Oracle accuracy (%)          | 76.14          | 75.70 |
| Best-fixed accuracy (%)      | 67.58          | 67.30 |
| Headroom (pp)                | 8.56           | 8.40  |
| D—A headroom (pp)            | −0.16          |       |
| Unadjusted bootstrap 95% CI  | [−0.83, +0.87] |       |
| One-sided sign-flip <i>p</i> | 0.647          |       |

**Table 9: Corrected primary analysis.** Means over 18 builder–seed cells on 1,169 questions; single-call is included in both the oracle and best-fixed comparators.

The diagonal forced/gated minus free/ungated contrast in single-call-inclusive oracle headroom is −0.16 pp, with cell contrasts ranging over −4.36 to +2.74 pp: the bundled screening policy does not improve mean headroom. The uncertainty estimates for this corrected analysis appear in Table 9; this mean comparison does not establish equivalence. Structural admission is therefore not evidence that a mechanism activates on relevant tasks or adds stable utility.

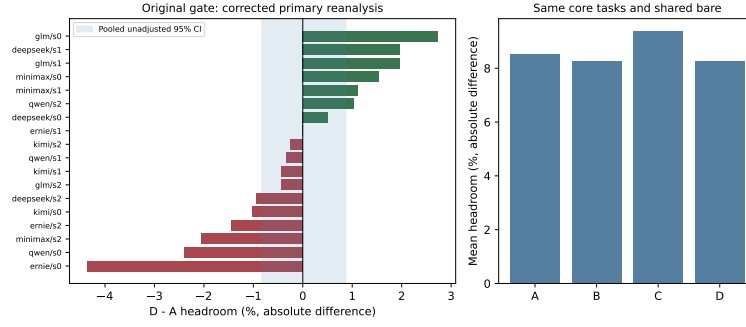


Figure 4: **Versioned corrected reanalysis.** Left: full-set D–A cell contrasts with the pooled 95% interval. Right: four-arm headroom on the shared core-400 tasks with a common single-call baseline. Headroom and contrast axes show percentage points.

On the shared core-400 tasks the gate contrast is  $-0.69$  pp, and the strategy and interaction secondaries are likewise inconclusive (Table 10); the fixed- $K$  mean contrast  $-0.31$  pp provides a population-size sensitivity check. The open-mechanism arm ran only on the development side and has no confirmatory estimand. These are all single-run contrasts, and the execution variability reported in the R2/R3 audits in Appendix F.3 (8.54% R2 verdict flips; 14.5% of tasks changing single-call correctness across three runs; a 6.50 pp same-code one-run gap) characterizes them as coverage opportunities rather than capability differences.

| Contrast    | Effect (pp) | 95% CI (pp)      | Raw $p$ | Holm $p$ |
|-------------|-------------|------------------|---------|----------|
| Gate        | $-0.69$     | $[-1.56, +0.20]$ | 0.026   | 0.079    |
| Strategy    | $+0.42$     | $[-0.32, +1.51]$ | 0.263   | 0.428    |
| Interaction | $-0.87$     | $[-2.83, +0.65]$ | 0.214   | 0.428    |

Table 10: **Corrected core-400 factorial reanalysis.** Gate is  $[(B-A) + (D-C)]/2$ , strategy  $[(C-A) + (D-B)]/2$ , interaction  $(D-C) - (B-A)$ ; raw and Holm-adjusted  $p$ -values.

## F.2 SAME-CORE HEADROOM DECOMPOSITION

Table 11 separates coverage, best-fixed accuracy, and headroom using the same core tasks and baseline records in every arm.

| Arm | $K_{\text{cand}}$ | $K_{\text{total}}$ | Single-call (%) | Best (%) | Oracle (%) | $H$ (%) |
|-----|-------------------|--------------------|-----------------|----------|------------|---------|
| A   | 5.78              | 6.78               | 63.25           | 66.00    | 74.51      | 8.51    |
| B   | 6.11              | 7.11               | 63.25           | 66.12    | 74.39      | 8.26    |
| C   | 5.61              | 6.61               | 63.25           | 65.64    | 75.01      | 9.38    |
| D   | 3.83              | 4.83               | 63.25           | 65.65    | 73.90      | 8.25    |

Table 11: **Same-core headroom decomposition.** All arms use identical 400 tasks and identical single-call outcomes.  $K_{\text{cand}}$  excludes single-call and  $K_{\text{total}} = K_{\text{cand}} + 1$ ; counts and accuracies are cell means. Headroom is oracle minus best-fixed accuracy, not realized routing gain.

## F.3 SENSITIVITY, REPEAT AUDITS, AND RECORDED COST

**Corrected K-matched and R2 replays.** Both replays use the same deterministic record ordering and shared database/within-database task draws as the corrected primary analysis, with seeds re-sampled within fixed builders (10,000 replicates). For K-matching, each A/B and C/D pair uses the smaller candidate count; every uniform candidate subset is enumerated exactly. Single-call stays in every subset, and best-fixed is recomputed inside each subset on every task draw. This is a size sensitivity analysis, not a causal mediation decomposition. Retaining all 18 cells, including two

with singleton A subsets, changes subset D–A from +0.42 to +0.81 pp (change +0.39 pp, 95% CI [-0.95, +1.63]). R2 subset headroom excludes single-call because R2 did not acquire a matched single-call run, so it is distinct from the single-call-inclusive primary estimand; all 18 cells are retained. These are instability measurements, not standard errors or a detectability floor for the mean treatment effect; they do not establish that every contrast is unchanged, or identify what fraction of between-harness disagreement is noise.

R2 acquired 70 harnesses on 400 tasks, selecting the first two callable admitted candidates per A/D cell where available. The two cache-off acquisition groups are kept separate despite a duplicated repeat label; program identity is checked against the cached runs. No matched single-call was acquired, so R2 headroom excludes single-call in both conditions. The corrected analysis retains singleton cells (their subset headroom is zero), yielding all 18 paired cells. There are 2391/28,000 verdict flips against cached outcomes (8.54%) and 1245/14,400 flips between the two D cache-off passes (8.65%). These observed replays do not identify future execution variance, the effect of caching alone, or stable complementarity over clones. Acquisition time and caching condition are not isolated.

**R3 single-call-only repeat audit.** R3 is a separate, smaller instability audit: three independent cache-off executions of the *single-call baseline only* (no generated harnesses) on the same core-400 tasks, same frozen target (GLM-5.3-Flash, temperature 0). 58 of 400 tasks (14.5%) changed single-call correctness in at least one of the three repeats; pairwise flip rates between repeat pairs are 39/400, 40/400, and 37/400 (9.75%, 10.0%, 9.25%). Treating the three single-call repeats as three pseudo-members (one execution each) gives a one-run oracle of 72.25% against a best-single-repeat accuracy of 65.75%, a descriptive 6.50 pp same-code oracle gap computed from the recorded official-scorer outcomes. R3 therefore measures the observed instability rate of the baseline itself under temperature-0 API nondeterminism. This is not a standard error or an effect-size floor for any contrast, but it sets the scale at which repeat controls become necessary for interpretation: a harness-versus-single-call difference of this magnitude cannot be attributed to capability without them.

| Sensitivity estimand      | Estimate (%) | 95% CI (%)     |
|---------------------------|--------------|----------------|
| K-matched B–A headroom    | −0.55        | [−1.53, +0.63] |
| K-matched D–C headroom    | −0.08        | [−1.26, +0.80] |
| K-matched gate mean       | −0.31        | [−1.07, +0.40] |
| R2 subset D–A (cached)    | +0.42        | [−0.61, +1.48] |
| R2 subset D–A (cache-off) | +0.81        | [−0.35, +1.84] |
| R2 change in subset D–A   | +0.39        | [−0.95, +1.63] |

Table 12: Corrected sensitivities on core-400, 18 cells. CIs are unadjusted. K-matched comparisons always retain single-call; R2 comparisons exclude single-call and are conditional on the observed acquisition groups.

**Recorded cost and missing billing data.** The same canonical core records contain 259,304 logged logical solver calls across 153,600 candidate–task records (Table 13). These calls include cache-served requests and are not provider-billed API requests. None of the inspected records has token usage fields; we do not impute token or monetary expenditure from call counts. Actual generation attempts, admitted candidate counts, and these execution counts therefore remain separate measurements. The population cost mean gives each builder–seed cell equal weight and excludes single-call; zero-candidate populations would contribute zero solver calls.

| Arm | Candidate-task records | Logged solver calls | Calls/task/population |
|-----|------------------------|---------------------|-----------------------|
| A   | 41,600                 | 67,258              | 9.34                  |
| B   | 44,000                 | 64,399              | 8.94                  |
| C   | 40,400                 | 75,604              | 10.50                 |
| D   | 27,600                 | 52,043              | 7.23                  |

Table 13: Descriptive canonical cost audit on core-400, 18 cells per arm. Logged logical calls are not billable requests or token counts. The last column averages candidate-population calls per task over cells.

#### F.4 ADMISSION SCOPE AND INSTRUMENT LIMITS

In the free arm, builders choose among four declarations: repair, vote, twostage, and plain. Vote requires multiple samples and selection, not necessarily three-way majority voting; twostage requires artifact consumption without prescribing a response interface. Plain declares no control-flow mechanism despite the prompt’s request for a control-flow change. Thus “free” means builder choice within a specified vocabulary, not an unrestricted interface.

The gate bundles structural screening with conformance verification. Independent development checks identified false rejections, request-attribution and response-interface defects, and unresolved conformance judgments. Later interface revisions and adjudication did not yield revised benchmark outcomes. The reported treatment therefore remains the implemented screening bundle; those development checks do not establish general semantic validity of the admission instrument.

**Generation-interface limits.** All 108 format-guard attempts across the 36 forced-arm cells failed neutral validity: 71 logs report unterminated strings, 36 no Python fence, and one a provider timeout. Synthetic examples reproduce truncation by the archived non-greedy code-fence extractor, but full builder messages and the exact version executed on each historical attempt were not retained. These observations document an upstream validity failure; they do not establish nonconformance of the intended mechanisms or attribute every rejection to the gate.